Asadito RG: guía para entender, probar y continuar el proyecto

Esta guía toma como referencia el archivo “Prácticas Profesionalizantes II - Criterios para los examenes.pdf” que compartiste. Sus criterios se usan para orientar el desarrollo; la aprobación depende de la evaluación docente y de tu dominio del proyecto. Las mejoras particulares del examen final todavía deben ser indicadas por la cátedra.

1. Qué problema resuelve

Una persona quiere comprar los cortes para una juntada y saber cuánto cuesta su lista completa en distintos comercios de Río Grande. La aplicación consulta precios publicados, permite elegir kilos y compara presupuestos por local. El resultado se descarga en PDF y se puede compartir como resumen de texto por WhatsApp.

Decisión de negocio: la comparación del presupuesto usa la MISMA lista en todos los locales. Un comercio sin precio de uno de los cortes no participa del ranking de totales. Un precio ausente no vale cero. Este diseño compara una compra completa en un solo comercio; combinar compras en varios locales es una mejora futura diferente.

No se publicaron precios de ejemplo como si fueran reales. Los 8 cortes y 13 locales iniciales vienen del proyecto original. El administrador debe cargar precios con fecha y confirmar su fuente. Los archivos Excel/CSV del repositorio no se importan automáticamente.

2. Correspondencia con los criterios de la materia

Criterio del documento	Implementación y evidencia	Lo que debés comprobar o explicar
1. Sesiones y autenticación	Spring Security, login por formulario, cookie JSESSIONID, logout, endpoint de sesión	Explicar por qué se usa sesión y cómo se renueva el ID al autenticar
2. Contraseñas seguras	BCrypt y columna passwordHash; nunca se devuelve el hash en la API	Diferenciar hash y cifrado; no agregar contraseñas de prueba al código
3. ABM principal	Productos, locales y precios: altas, edición y baja desde panel ADMIN	Demostrar las tres operaciones en cada entidad
4. Bajas lógicas	Campo activo; las bajas no llaman delete del repositorio	Mostrar que el registro permanece y desaparece de consultas públicas
5. Organización	config, controllers, dtos, services, modelos, repositorios; frontend separado en assets	Seguir el recorrido de una solicitud desde la pantalla hasta la BD
6. Entidades y POO	Usuario; Locales y Productos relacionados por Precios	Justificar cardinalidades, identidad y responsabilidades
7. Validaciones Java	Bean Validation, control de relaciones activas, errores JSON	Probar precio negativo, fecha futura, kilos inválidos y campos vacíos
8. Integración frontend/backend	Fetch con cookie y CSRF; tablas, formularios, ranking y PDF reales	Probar sin CDN y revisar mensajes ante errores
9. Compila y funciona	Build, pruebas de integración y comprobación funcional	Ejecutarlo en tu computadora antes del examen
10. Idea original	Comparador, calculadora en kilos y PDF por local	Mostrar una lista idéntica en dos comercios con resultados distintos
11. Dominio del proyecto	Esta guía y ejercicios de modificación	Leer y practicar: la guía no sustituye comprender el código

No se afirma que estos cambios garanticen la aprobación ni que cubran requisitos adicionales del final aún no comunicados.

3. Ponerlo en marcha

Base de datos y configuración

Seguí el README. Las variables importantes son:

• SPRING_DATASOURCE_URL: JDBC de la base PostgreSQL.

• SPRING_DATASOURCE_USERNAME y SPRING_DATASOURCE_PASSWORD: credenciales locales, fuera de Git.

• ASADITO_ADMIN_EMAIL y ASADITO_ADMIN_PASSWORD: creación inicial opcional del administrador. Nunca hay un ADMIN público por defecto.

• PORT: puerto HTTP, por defecto 8080.

• SESSION_COOKIE_SECURE: false para HTTP local, true cuando el despliegue usa HTTPS.

El bootstrap no cambia la contraseña de un administrador existente ni convierte usuarios USER en ADMIN. Si el email ya está registrado como USER, elegí un email nuevo para el administrador o realizá una operación administrativa controlada. No agregues un endpoint público para elegir roles.

En una base existente: detené la aplicación, hacé un respaldo con pg_dump y ejecutá docs/migracion-v2.sql con psql usando ON_ERROR_STOP. El script añade activo con valor true y convierte precios a numeric(14,2), redondeando registros anteriores a dos decimales. El redondeo de datos históricos debe revisarse antes de aplicarlo a una base real. En bases nuevas, Hibernate crea el esquema.

La migración es idempotente. No borra tablas ni elimina datos. ddl-auto=update se conserva para facilitar desarrollo, pero no reemplaza un sistema de migraciones para producción.

Datos iniciales

1. Iniciá la aplicación con las variables del primer administrador.

2. Ingresá con esa cuenta.

3. Abrí Administrar y cargá el precio por kilo de un corte en dos locales.

4. La fecha no puede ser futura. Cuando hay varios precios de la misma fecha para un corte/local, se usa el registro con mayor ID.

5. Retirá las variables de bootstrap cuando ya tengas el administrador creado.

El catálogo arranca sin precios. Ver un estado vacío es correcto. No implica que la importación automática esté funcionando.

4. Cómo funciona la seguridad

SecurityConfig arma la cadena de filtros de Spring Security. Las consultas del catálogo y el cálculo son públicos. Los endpoints de administración requieren ROLE_ADMIN. El PDF requiere un usuario autenticado. La interfaz oculta el panel cuando corresponde, pero la protección real está en el servidor.

El registro recibe email y password. El email se normaliza y tiene unicidad en la base. El servidor asigna USER; rechaza campos no reconocidos como rol. BCrypt aplica sal y costo de trabajo. La contraseña se valida antes de codificarla: mínimo 10 caracteres y máximo 72 bytes UTF-8, límite relevante de BCrypt. Un carácter acentuado puede ocupar más de un byte.

El login es POST /api/auth/login y recibe email y password como formulario URL-encoded. Spring Security verifica la contraseña y administra la sesión, incluida protección contra fijación de sesión. La cookie es HttpOnly y SameSite=Lax. La sesión expira luego de 30 minutos de inactividad. Logout es POST y elimina la sesión y cookie.

CSRF permanece habilitado. El frontend obtiene token y nombre de cabecera en GET /api/auth/csrf, envía la cabecera en cada POST/PUT/DELETE y renueva el token después de login y logout. La cookie se envía con credentials same-origin. No se guardan contraseñas ni tokens de autenticación en localStorage.

Un 401 significa que necesitás autenticarte. Un 403 puede indicar falta de permiso o CSRF vencido: recargá la página antes de repetir. La interfaz no reintenta automáticamente una escritura, para no duplicar operaciones.

Antes de exponer el sitio a Internet: HTTPS, cookie Secure, credenciales de BD reales, límites de intentos de login y registro, recuperación de contraseña y revisión de versiones soportadas. El proyecto conserva Spring Boot 2.7 y javax para ser compatible con la base actual; planificá una migración a una versión mantenida sin mezclarla con cambios visuales.

5. Recorrido del catálogo y las bajas lógicas

CatalogController recibe las solicitudes y valida los DTO. CatalogService contiene las operaciones, busca relaciones activas y usa repositorios JPA. Los DTO de entrada limitan lo que el cliente puede mandar; las respuestas de catálogo son proyecciones simples y no serializan entidades con ciclos.

Cada precio pertenece a un local y un producto. Es un registro con fecha, importe y estado. Dar de baja un local o producto oculta también sus precios en las consultas públicas sin borrar el historial. Dar de baja el precio más reciente permite que vuelva a utilizarse el registro activo anterior: es una regla explícita de esta versión, importante para explicar en el examen. Si el negocio exige que una baja signifique “sin precio vigente”, necesitarás modelar vigencia o cancelación en lugar de volver al anterior.

Los IDs identifican los locales. Dos comercios con el mismo nombre no se mezclan. La selección del último precio ordena fecha e ID para resolver empates de forma determinista.

No hay restauración de bajas desde la interfaz ni panel de auditoría. Agregarlos es un buen ejercicio posterior, con permisos administrativos y pruebas de que no resuciten datos accidentalmente.

6. Cómo se calcula el presupuesto

El navegador manda localId y una lista de productoId/cantidad. No manda precios ni total. PresupuestoService obtiene los precios activos más recientes del servidor. Así, modificar el JavaScript no permite falsificar el total del PDF.

El dinero usa BigDecimal y numeric(14,2). Cada subtotal se calcula como precio por kilo por cantidad y se redondea a dos decimales con HALF_UP; el total suma los subtotales ya redondeados. Ejemplo: 10000,01 por 1,5 kg da 15000,015 y se informa 15000,02. No uses float para dinero.

Las cantidades admiten entre 0,001 y 1000 kg, hasta tres decimales. Se rechazan cortes duplicados, listas vacías, más de 100 ítems, relaciones inexistentes y cortes sin precio en el local. La pantalla permite cero para dejar un corte fuera de la lista; solo envía los positivos.

Si cambia la lista o se actualiza el catálogo se invalida el presupuesto mostrado y hay que recalcular. Los locales con listas incompletas nunca se presentan como opciones más baratas. Los precios de fechas anteriores no se marcan como ofertas actuales: la fecha se muestra explícitamente.

Límite actual: se realizan solicitudes por comercio y se consulta el catálogo completo. Para cientos de locales, implementá un cálculo comparativo único en el servidor, filtros, paginación, índices por local/producto/fecha e instrucciones sobre antigüedad máxima de precios.

7. PDF y WhatsApp

PdfService usa PDFBox en el backend. Incluye comercio, dirección, kilos, precio por kilo, subtotal, fechas, moneda, total y aviso de presupuesto estimativo. Tiene paginación. Usa una fuente PDF incorporada compatible con texto español; reemplaza caracteres no soportados y recorta líneas demasiado largas para evitar que salgan de la hoja. Si necesitás nombres en cualquier idioma, incorporá una fuente Unicode con licencia apropiada y ajuste de líneas.

El PDF se genera con precios vigentes al pedirlo; si otro administrador actualiza precios entre el cálculo visual y la descarga, el documento puede reflejar la actualización. Una mejora recomendable es guardar un presupuesto inmutable con ID, usuario, fecha y detalle de precios. Usá el mismo ID para vista previa, PDF e historial, y controlá la propiedad del usuario en cada consulta.

Compartir resumen por WhatsApp abre wa.me con texto codificado. El usuario elige conversación y confirma el envío. No sube el documento ni envía mensajes automáticamente. No se realizó ningún envío

real durante las pruebas.

Compartir el PDF usa la API Web Share de archivos cuando está disponible, normalmente en móviles y contextos seguros. El usuario elige WhatsApp si aparece en el menú. Si no hay soporte, debe adjuntar el archivo descargado manualmente. Un enlace wa.me no puede adjuntar por sí solo un archivo local del navegador.

Ruta para WhatsApp Business en una etapa posterior

1. Definir si hace falta enviar desde una cuenta empresarial o si el envío manual cubre el uso real.

2. Si se necesita automatización, crear y verificar la configuración de Meta/WhatsApp Business correspondiente. No inventar claves ni guardar tokens en el frontend.

3. Crear presupuestos persistentes y verificar usuario propietario y consentimiento del destinatario.

4. Implementar un servicio backend que suba el PDF como media y envíe el mensaje de documento usando la versión vigente de la API.

5. Revisar las reglas actuales de plantillas, ventanas de conversación y aceptación de mensajes; pueden cambiar.

6. Guardar secretos solo en la configuración del servidor. Añadir límites, auditoría, idempotencia, cola/reintentos y webhook validado para estados.

7. Probar con números autorizados del entorno de prueba antes de habilitar envíos reales.

Esta integración automática queda pendiente; no es una función simulada ni se declara implementada.

8. Mapa de endpoints

Ruta	Acceso	Uso
GET /api/auth/csrf	Público	Token CSRF de la sesión
GET /api/auth/me	Público	Estado de la sesión y rol administrativo
POST /api/auth/registro	Público + CSRF	Crear USER
POST /api/auth/login	Público + CSRF	Autenticar formulario
POST /api/auth/logout	Sesión + CSRF	Invalidar sesión
GET /api/productos, /api/locales, /api/precios	Público	Catálogo activo
GET /api/precios/ultimos	Público	Último precio activo por corte/local
POST /api/admin/productos, /locales, /precios	ADMIN + CSRF	Alta
PUT /api/admin/productos/{id}, /locales/{id}, /precios/{id}	ADMIN + CSRF	Edición
DELETE /api/admin/{tipo}/{id}	ADMIN + CSRF	Baja lógica
POST /api/presupuestos/calcular	Público + CSRF	Validar y calcular
POST /api/presupuestos/pdf	Autenticado + CSRF	Descargar PDF recalculado

Ejemplo de cuerpo para calcular (reemplazá los IDs por los de tu catálogo):

```
{"localId":1,"items":[{"productoId":1,"cantidad":1.5}]}
```

Ejemplo de alta de precio:

```
{"localId":1,"productoId":1,"precio":12500.50,"fecha":"2026-11-02"}
```

Usá una fecha válida y no futura en tu prueba. Los errores previsibles devuelven JSON con message y, cuando corresponde, fields. Estados: 400 formato/validación; 401 autenticación; 403 permisos/CSRF; 404 registro inexistente o inactivo; 409 conflicto; 422 falta de precio para completar una lista.

9. Pruebas para ensayar antes de presentar

Automáticas: ejecutar bash gradlew test o el helper cloud del README. H2 permite probar sin depender de PostgreSQL. Las pruebas de MockMvc atraviesan filtros de seguridad, validadores, servicios y persistencia; las pruebas PDF abren el documento y verifican contenido y paginación. H2 no reemplaza comprobar la migración y el arranque en PostgreSQL.

Ensayo manual recomendado:

1. Abrir en una ventana privada: comparar sin cuenta, intentar descargar PDF y observar el pedido de ingreso.

2. Registrar USER; cerrar sesión; probar una contraseña incorrecta; volver a ingresar; confirmar que no aparece administración.

3. Con ADMIN, crear, editar y dar de baja un producto, local y precio. Revisar la fila conservada en la base.

4. Publicar un corte en dos locales: 10000 y 12000 por kilo. Con 1,5 kg, comprobar 15000 y 18000. El primero debe encabezar el ranking.

5. Agregar un segundo corte sin precio en uno de los locales: ese comercio debe aparecer incompleto, sin total competitivo.

6. Intentar datos vacíos, precio negativo, fecha futura, coordenada 91 de latitud, cantidad negativa y cuatro decimales. Verificar rechazo en Java aunque se omita el HTML.

7. Descargar PDF y comparar sus cantidades e importes con los precios del servidor. Abrirlo en un lector real.

8. Abrir el enlace WhatsApp y revisar el texto sin enviarlo. En un móvil compatible, probar el menú de compartir archivo.

9. Probar ancho de pantalla de 390 px, navegación por teclado, cierre de diálogos y etiquetas de formularios.

10. Cerrar sesión y confirmar que los endpoints protegidos no aceptan la sesión anterior. Reiniciar el servidor y confirmar que la cuenta y catálogo persisten.

La prueba funcional de navegador realizada en el entorno usa Chromium y una base PostgreSQL efímera independiente; no crea precios ficticios en tu catálogo de desarrollo.

10. Ejercicios para dominar el código

• Seguí el clic en “Calcular”: evento JS, fetch, CSRF, controlador, DTO, servicio, repositorio y respuesta. Explicalo sin leer la guía.

• Cambiá el máximo permitido a 500 kg. ¿Qué validación backend y qué atributo HTML/validación JS deben cambiar? Agregá una prueba de 501.

• Agregá teléfono opcional al local. Modificá entidad, migración, DTO de entrada, validación, proyección y formulario. No confundas teléfono con credencial.

• Agregá restauración de productos dados de baja solo para ADMIN. Definí cómo afecta sus precios y escribí una prueba.

• Probá dos locales con el mismo nombre. Explicá por qué los IDs mantienen presupuestos independientes.

• Simulá la baja del último precio y explicá qué registro anterior se selecciona.

• Justificá por qué una lista incompleta no debe ganar por total menor y por qué el backend recalcula el PDF.

• Explicá por qué ocultar el botón “Administrar” no es un control de autorización suficiente.

11. Pendientes priorizados para continuar por tu cuenta

Prioridad 1, antes de una entrega real: conseguir precios actuales con fuente/fecha comprobables, cargar datos representativos, ensayar todas las operaciones y preparar explicación del modelo. Rotar cualquier credencial real que haya estado en el archivo application.properties o en su historial Git: borrarla del archivo actual no la elimina del historial.

Prioridad 2, calidad y evolución: migraciones versionadas con Flyway/Liquibase y ddl-auto=validate; optimizar relaciones EAGER y consultas del catálogo; agregar índices y paginación; auditoría de cambios, vigencia de precios y recuperación de bajas; pruebas adicionales para concurrencia de administradores y fallos de red.

Prioridad 3, producto: presupuestos persistentes por usuario, historial y repetición de listas, filtros por zona, compra combinada entre comercios y un importador Excel/CSV con vista previa, validación y detección de duplicados. Los archivos heredados requieren revisar formato y procedencia, no cargarlos ciegamente.

Prioridad 4, despliegue: actualizar Spring Boot y dependencias a versiones mantenidas; revisar la migración javax a jakarta; HTTPS, sesiones seguras, protección contra abuso, backup/restauración probados, gestión de errores operativos y monitoreo. El código base de desarrollo no debe presentarse como una plataforma lista para producción.

Prioridad 5, integración comercial: WhatsApp Business solo si aporta valor frente al envío manual, con las condiciones y controles explicados antes.

Los binarios y recursos de build/ ya estaban versionados. Los cambios nuevos no los sobrescriben durante la validación cloud. Como limpieza futura independiente, revisá retirar esos archivos del índice con git rm --cached, manteniendo las fuentes y comprobando la compilación limpia. .gitignore evita nuevas salidas, pero no desversiona archivos existentes.